Assignment - 8

1. WAP to demonstrate the use of forks, exec() and getpid() system calls.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
pid_t pid;
pid = fork();

if (pid < 0)
{
printf("fork failed\n");
return 1;
}
else if (pid == 0)
{
printf("Child process :\n");
printf("PID = %d\n", getpid());
exec...("/bin/ls","ls",NULL);
printf("Exec Failed");
}
else
{
printf("Parent process :\n");
printf("PID = %d\n", getpid());
printf("Child PID = %d\n", pid);
}
return 0;
}
```

OUTPUT

Parent process :

PID : 2451

Child process :

PID : 2452

Total 2

---

2. WAP to demonstrate inter process communication.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main()
{
int fd[2];
pid_t pid;
char write_msg[] = "Hello from parent";
char read_msg[50];

if (pipe(fd) == -1)
{
printf("pipe failed\n");
return 1;
}

pid = fork();

if (pid < 0)
{
printf("fork failed\n");
return 1;
}
else if (pid > 0)
```

```
{
close(fd[0]);
write(fd[1], write_msg, strlen(write_msg) + 1);
printf("Parent sent: %s\n", write_msg);
close(fd[1]);
}
else
{
close(fd[1]);
read(fd[0], read_msg, sizeof(read_msg));
printf("Child received: %s\n", read_msg);
close(fd[0]);
}

read(fd[0], read_msg, sizeof(read_msg));
printf("Child received: %s\n", read_msg);
close(fd[0]);
return 0;
}
```

OUTPUT

Parent sent : Hello from parent.
Child received : Hello from parent.